

UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

[Russ Tedrake](#)

© Russ Tedrake, 2024

Last modified 2024-11-11.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

CHAPTER 18

System Identification

 Launch in Deepnote

My primary focus in these notes has been to build algorithms that design of analyze a control system *given a model* of the plant. In fact, we have in some places gone to great lengths to understand the structure in our models (the structure of the manipulator equations in particular) and tried to write algorithms which exploit that structure.

Our ambitions for our robots have grown over recent years to where it makes sense to question this assumption. If we want to program a robot to fold laundry, spread peanut butter on toast, or make a salad, then we should absolutely not assume that we are simply given a model (and the ability to estimate the state of that model). This has led many researchers to focus on the "model-free" approaches to optimal control that are popular in reinforcement learning. But I worry that the purely model-free approach is "throwing the baby out with the bathwater". We have fantastic tools for making long-term decisions given a model; the model-free approaches are correspondingly much much weaker.

So in this chapter I would like to cover the problem of learning a model. This is far from a new problem. The field of "system identification" is as old as controls itself, but new results from machine learning have added significantly to our algorithms and our analyses, especially in the high-dimensional and finite-sample regimes. But well before the recent machine learning boom, system identification as a field had a very strong foundation with thorough statistical understanding of the basic algorithms, at least in the asymptotic regime (the limit where the amount of data goes to infinity). Machine learning theory has brought a wealth of new results in the online optimization and finite-sample regimes. My goal for this chapter is to establish these foundations, and to provide some pointers so that you can learn more about this rich topic.

18.1 PROBLEM FORMULATION

18.1.1 Equation error vs simulation error



Our problem formulation inevitably begins with the data. In practice, if we have access to a physical system, instrumented using digital electronics, then we have a system in which we can apply input commands, $\mathbf{u}_n$, at some discrete rate, and measure the outputs, $\mathbf{y}_n$ of the system at some discrete rate. We normally assume these rates are fixed, and often attempt to fit a state-space model of the form

$$\mathbf{x}[n+1] = f_\alpha(\mathbf{x}[n], \mathbf{u}[n]), \quad \mathbf{y}[n] = g_\alpha(\mathbf{x}[n], \mathbf{u}[n]), \quad (1)$$

where I have used α again here to indicate a vector of parameters. In this setting, a natural formulation is to minimize a least-squares estimation objective:

$$\min_{\alpha, \mathbf{x}[0]} \sum_{n=0}^{N-1} \|\mathbf{y}[n] - \mathbf{y}_n\|_2^2, \quad \text{subject to (1).}$$

I have written purely deterministic models to start, but in general we expect both the state evolution and the measurements to have randomness. Sometimes, as I have written, we fit a deterministic model to the data and rely on our least-squares objective to capture the errors; more generally we will look at fitting stochastic models to the data.

We often separate the identification procedure into two parts, in which we first estimate the state $\hat{\mathbf{x}}_n$ given the input-output data $\mathbf{u}_n, \mathbf{y}_n$, and then focus on estimating the state-evolution dynamics in a second step. The dynamics estimation algorithms fall into two main categories:

- *Equation error* minimizes only the one-step prediction error:

$$\min_{\alpha} \sum_{n=0}^{N-2} \|f_\alpha(\hat{\mathbf{x}}_n, \mathbf{u}_n) - \hat{\mathbf{x}}_{n+1}\|_2^2.$$

- *Simulation error* captures the long-term prediction error:

$$\min_{\alpha} \sum_{n=1}^{N-1} \|\mathbf{x}[n] - \hat{\mathbf{x}}_n\|_2^2, \quad \text{subject to } \mathbf{x}[n+1] = f_\alpha(\mathbf{x}[n], \mathbf{u}_n), \mathbf{x}[0] = \hat{\mathbf{x}}_0,$$

The equation-error formulations often result in much more tractable optimization problems, but unfortunately we will see that optimizing the one-step error can still result in arbitrarily large simulation errors. Therefore, we generally consider the simulation error to be the true objective we hope to optimize, and the equation error only as a potentially useful surrogate.

Within this framework of model learning, we will face all of the standard questions from supervised learning about sample efficiency and generalization.

18.1.2 Online optimization

Theoretical machine learning has brought a number of fresh perspectives to this old problem of system identification. Perhaps most relevant is the perspective of online optimization [1, 2, 3]. In addition to changing the formulation to considering streaming online data (as opposed to offline, batch analysis), this field has

(re)popularized the notion of an old idea from decision theory: using *regret* as the loss function for identification.

In the regret formulation, we acknowledge that the class of parameterized models that we are searching over can likely not perfectly represent the input-output data. Rather than score ourselves directly on the prediction error, we can score ourselves relative to the predictions that would be made by best parameters in our model class. For equation-error formulations, the regret R at step N would take the form

$$R[N] = \sum_{n=0}^{N-2} \|f_{\alpha}(\hat{\mathbf{x}}_n, \mathbf{u}_n) - \hat{\mathbf{x}}_{n+1}\|_2^2 - \min_{\alpha^*} \sum_{n=0}^{N-2} \|f_{\alpha^*}(\hat{\mathbf{x}}_n, \mathbf{u}_n) - \hat{\mathbf{x}}_{n+1}\|_2^2.$$

The goal is to produce an online learning algorithm to select α at each step in order to drive the regret to zero. This turns out to be an important distinction, as we will see below.

18.1.3 Learning models for control

So far we've described the problem purely as capturing the input-output behavior of the system. If our goal is to use this model for control, then that might not be quite the right objective for a number of reasons.

First of, predicting the observations might be *more* than we need for making optimal decisions; [4] tells that story nicely. Imagine you are trying to swing-up and balance a cart-pole system, where the only sensor is a camera. The background images are irrelevant; we know that only the state of the cart-pole should matter. If there was a movie playing on a screen in the background, you don't want to spend all of the representational power of your model predicting the next frame of the movie, at the cost of not predicting as well the next state of the cart-pole. These challenges have led to a nice active area of research on learning *task-relevant* models/representations; I'll devote a section to it below after we cover the basics.

There are other considerations, as well. To be useful for control, we'll need the state of our model to be observable (from online observations) and controllable. There may also be trade-offs in model complexity. If we learn a just linear model of the dynamics then we have very powerful control design tools available, but we might not capture the rich dynamics of the world. If we learn too complex of a model, then our online planning and control techniques might become a bottleneck. [5] makes that point nicely. At the time of this writing, I think it's broadly fair to say that deep learning models are able to describe incredibly rich dynamics, but that our control tools for making decisions with these models are still painfully weak.

18.2 PARAMETER IDENTIFICATION FOR MECHANICAL SYSTEMS

My primary focus throughout these notes is on (underactuated) mechanical systems, but when it comes to identification there is an important distinction to make. For some mechanical systems we know the structure of the model, including number of state variables and the topology of the kinematic tree. Legged robots like Spot or Atlas are good examples here -- the dynamics are certainly nontrivial, but the general form of the equations are known. In this case, the task of identification is really the task of estimating the parameters in a structured model. That is the subject of this section.

The examples of folding laundry or making a salad fall into a different category. In those examples, I might not even know a priori the number of state variables needed

to provide a reasonable description of the behavior. That will force a more general examination of the the identification problem, which we will explore in the remaining sections.

Let's start with the problem of identifying a canonical underactuated mechanical system, like an Acrobot, Cart-Pole or Quadrotor, where we know the structure of the equations, but just need to fit the parameters. We will further assume that we have the ability to directly observe all of the state variables, albeit with noisy measurements (e.g. from joint sensors and/or inertial measurement units). The stronger [state estimation algorithms](#) that we will discuss soon assume a model, so we typically do not use them directly here.

Consider taking a minute to review the [example of deriving the manipulator equations for the double pendulum](#) before we continue.

18.2.1 Kinematic parameters and calibration

We can separate the parameters in the multibody equations again into kinematic parameters and dynamic parameters. The kinematic parameters, like link lengths, describe the coordinate transformation from one joint to another joint in the kinematic tree. It is certainly possible to write an optimization procedure to calibrate these parameters; you can find a fairly thorough discussion in e.g. Chapter 11 of [6]. But I guess I'm generally of the opinion that if you don't have accurate estimates of your link lengths, then you should probably invest in a tape measure before you invest in nonlinear optimization.

One notable exception to this is calibration with respect to joint offsets. This one can be a real nuisance in practice. Joint sensors can slip, and some robots even use relative rotary encoders, and rely on driving the joint to some known hard joint limit each time the robot is powered on in order to obtain the offset. I've worked on one advanced robot that had a quite elaborate and painful kinematic calibration procedure which involve fitting additional hardware over the joints to ensure they were in a known location and then running a script. Having an expensive and/or unreliable calibration procedure can put a damper on any robotics project.

Example 18.1 (Acrobot balancing with calibration error)

Small kinematic calibration errors can lead to large steady-state errors when attempting to stabilize a system like the Acrobot. I've put together a simple notebook to show the effect here:

 [Launch in Deeptime](#)

Our tools from robust / stochastic control are well-suited to identifying (and bounding / minimizing) these sensitivities, at least for the linearized model we use in LQR.

The general approach to estimating joint offsets from data is to write the equations with the joint offset as a parameter, e.g. for the [double pendulum](#) we would write the forward kinematics as:

$$\mathbf{p}_1 = l_1 \begin{bmatrix} \sin(\theta_1 + \bar{\theta}_1) \\ -\cos(\theta_1 + \bar{\theta}_1) \end{bmatrix}, \quad \mathbf{p}_2 = \mathbf{p}_1 + l_2 \begin{bmatrix} \sin(\theta_1 + \bar{\theta}_1 + \theta_2 + \bar{\theta}_2) \\ -\cos(\theta_1 + \bar{\theta}_1 + \theta_2 + \bar{\theta}_2) \end{bmatrix}.$$

We try to obtain independent measurements of the end-effector position (e.g. from motion capture, from perhaps some robot-mounted cameras, or from some mechanical calibration rig) with their corresponding joint measurements, to obtain data points of the form $\langle \mathbf{p}_2, \theta_1, \theta_2 \rangle$. Then we can solve a small nonlinear optimization problem to estimate the joint offsets to minimize a least-squares residual.

If independent measurements of the kinematics are not available, it is possible to estimate the offsets along with the dynamic parameters, using the trigonometric identities, e.g. $s_{\theta+\bar{\theta}} = s_{\theta}c_{\bar{\theta}} + c_{\theta}s_{\bar{\theta}}$, and then including the $s_{\bar{\theta}}, c_{\bar{\theta}}$ terms (separately) in the "lumped parameters" we discuss below.

18.2.2 Estimating inertial parameters (and friction)

Now let's think about estimating the inertial parameters of a multibody system. We've been writing the manipulation equations in the form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \text{friction, etc.} \quad (2)$$

Each of the terms in this equation can depend on the parameters α that we're trying to estimate. But the parameters enter the multibody equations in a particular structured way: *the inverse dynamics equations are affine in the inertial parameters* [7]. Notably, the forward dynamics (solving for $\ddot{\mathbf{q}}$) involves a $\mathbf{M}^{-1}(\mathbf{q})$; these dynamics are *not* affine in the parameters. Generic system identification approaches which treat the multibody dynamics as $\mathbf{x}[n+1] = \mathbf{f}(\mathbf{x}[n], \mathbf{u}[n])$ will not be leveraging this convexity directly.

To proceed with the inverse dynamics formulation, let us first choose a parameterization for the inertias. [8] shows that the natural parameterization of the inertial terms for a particular rigid body, B , are in terms of the *density-weighted moment matrices*:

$$\begin{aligned} m_B &= \iiint_{V_B} \rho(\mathbf{x}) dV \in \mathbb{R} \\ \mathbf{h}_B &= \iiint_{V_B} \mathbf{x} \rho(\mathbf{x}) dV \in \mathbb{R}^3 \\ \boldsymbol{\Sigma}_B &= \iiint_{V_B} \mathbf{x} \mathbf{x}^T \rho(\mathbf{x}) dV \in S_3, \end{aligned}$$

where V_B is the volume of the geometry in body B , the integral notation is for the 3D volume integral, and S_3 is the group of 3x3 (real) symmetric matrices. m_B is the total mass of the body, $\mathbf{h}_B$ is simply the mass times the position of the center of mass, and $\boldsymbol{\Sigma}_B$ is closely related to the more standard form of the inertia matrix. With the integral using x represented in the body frame, B , we can extract ${}^B\mathbf{p}^{B_{com}} = \frac{1}{m} \mathbf{h}_B$ and $\mathbf{I}^{B/B_0} = \text{tr}(\boldsymbol{\Sigma}_B) \mathbb{I}_{3 \times 3} - \boldsymbol{\Sigma}_B$. Since $\boldsymbol{\Sigma}_B$ is symmetric, we can summarize all of the inertial parameters for body B in 10 real numbers:

$$\alpha_B = [m_B, \mathbf{h}_B^T, \text{vec}_{\text{sym}}(\boldsymbol{\Sigma}_B)] \in \mathbb{R}^{10},$$

where vec_{sym} takes the 6 upper triangular elements of the 3x3 symmetric matrices into a vector. Naturally, if we aim to estimate the parameters of many bodies simultaneously, we can compose these into a single parameter vector α .

Remarkably, the inverse dynamics are affine in these parameters. This means that the manipulator equations above can be refactored into the form

$$\mathbf{W}(\mathbf{q}_n, \dot{\mathbf{q}}_n, \ddot{\mathbf{q}}_n, \mathbf{u}_n)\alpha_B + \mathbf{w}_0(\mathbf{q}_n, \dot{\mathbf{q}}_n, \ddot{\mathbf{q}}_n, \mathbf{u}_n) = 0. \quad (3)$$

We sometimes refer to the $\mathbf{W}$ which is assembled by vertically stacking these equations evaluated at all time samples, n , as the "data matrix". Given data trajectories of positions, $\mathbf{q}_n$, and torque measurements, $\mathbf{u}_n$, we can estimate $\dot{\mathbf{q}}_n$ and $\ddot{\mathbf{q}}_n$ with (acausal) filtering. Then we can estimate the parameters using least-squares estimation. As such, we can leverage all of the strong results and variants from linear estimation. For instance, we can add terms to regularize the estimate (e.g. to stay close to an initial guess), and we can write efficient recursive estimators for optimal online estimation of the parameters using recursive least-squares. My favorite recursive least-squares algorithm uses incremental QR factorization[9].

Not all values of α_B describe physically-valid inertias. For instance, we know that m_B has to be positive. There are constraints on the inertia matrix itself. [8] showed that these constraints are exactly transcribed as positive-definiteness of the "pseudo-inertia":

$$\mathbf{J}(\alpha_B) := \begin{bmatrix} \Sigma_B & \mathbf{h}_B \\ \mathbf{h}_B^T & m_B \end{bmatrix} \succ 0.$$

If we know the geometry of the body, B , that we are estimating, then we can optionally write additional convex constraints on these parameters. For instance, we know that the center of mass must lie inside the convex hull of the geometry:

$${}^B\mathbf{p}^{Bcom} \in \text{conv}(V_B) \Rightarrow \mathbf{h}_B \in m_B \text{conv}(V_B).$$

This non-negative scaling of a convex set is still a convex set (known as the "perspective" of the set). [8] also gives a convex constraint on the second-moment matrix based on e.g. the minimum-volume Loewner-John ellipsoid containing the known geometry.

Do these additional constraints matter? In the limit of infinite data, then one might hope that the least-squares estimator would obtain the correct values without the constraints. But there are two foils to this ideal. First, we often wish to operate in a relatively low-data regime. Second, it is important to understand that *not all of the inertial parameters will be identifiable*. For instance, if you imagine a robotic arm bolted to a table where the first link is a rotation about the z axis. Then no amount of data obtained from this robot will give any information about the moments of inertia of the first link about the x and y axes. Naively including these parameters in the estimation problem could not only lead to poor estimates of the unidentifiable parameters, but can even render the physical-validity constraints on the inertial parameters vacuous.

There are two general approaches to dealing with the unidentifiable parameters. First, we can exclude them from the optimization. There are a number of algorithms for symbolically extracting the identifiable parameters, but it is more common (and likely just as effective) to simply evaluate the null-space of a sufficiently rich data matrix, $\mathbf{W}$, to find the unidentifiable parameters by using, e.g., $\mathbf{R}_1\alpha_l$ from the QR factorization:

$$\mathbf{W} = [\mathbf{Q}_1 \quad \mathbf{Q}_2] \begin{bmatrix} \mathbf{R}_1 \\ 0 \end{bmatrix}, \quad \Rightarrow \quad \mathbf{W}\alpha_B = \mathbf{Q}_1(\mathbf{R}_1\alpha_B) = \mathbf{Q}_1\bar{\alpha}_B,$$

where $\bar{\alpha}_B$ are the identifiable parameters. To implement the physical-validity constraints, we would need to use estimates of the unidentifiable parameters. Alternatively, one can formulate the optimization directly on the full parameters and add a regularization term which (ideally) operates in the nullspace of the primary objective to drive the unidentifiable parameters towards a prior estimate. [10] gives a thorough treatment of this approach and provides a convex formulation of a geometrically consistent regularizer, which they call the *entropic divergence*.

The least-squares inverse-dynamics formulation also allows one to simultaneously fit a number of other relevant dynamic parameters which enter linearly into the equations. These include reflected inertial (as a diagonal matrix times $\ddot{\mathbf{q}}$), joint friction (as a constant torque multiplied by the sign of $\dot{\mathbf{q}}$), and damping (as a linear multiple of $\dot{\mathbf{q}}$).

Allow me to mention one subtle point about numerical scaling. All of the equations in the least-squares inverse-dynamics formulation (3) have units of torque. But the torques used on the base link of a robotic arm might be numerically much larger than the torques corresponding to the more distal joints. [10] addresses this by solving the unconstrained least-squares problem once, then taking the error covariance of this solution into the objective function for the constrained least-squares optimization. This serves to make the least-squares objective coordinate independent.

Example 18.2 (Inertial identification for a Kuka iiwa)

Solving the least-squares optimizations described here subject to convex (semidefinite) constraints requires solving small semidefinite programs. There has been some nice work on parameterizations which lead to unconstrained, smooth, but nonconvex, optimization problems which can be solved with e.g. gradient descent [11]. In some cases, one can still guarantee that all local minima are global minima. But my impression from talking to the authors is that numerics of the semidefinite programming formulation can be notable better, and that it should be preferred if SDP solvers are available.

18.2.3 Simultaneous kinematic and inertial identification via lumped parameters.

Although it is highly recommended to separate kinematic and inertial estimation, some amount of joint estimation is possible when needed. We know that the inverse dynamics are affine in the inertial parameters. But it turns out they can also be written as affine in the "lumped parameters": nonlinear combinations of inertial and kinematic which do not depend on the data. I feel this is best explained with a few simple examples.

Example 18.3 (Lumped parameters for the simple pendulum)

The now familiar equations of motion for the simple pendulum are

$$ml^2\ddot{\theta} + b\dot{\theta} + mgl \sin \theta = \tau.$$

For parameter estimation, we will factor this into

$$\begin{bmatrix} \ddot{\theta} & \dot{\theta} & \sin \theta \end{bmatrix} \begin{bmatrix} ml^2 \\ b \\ mgl \end{bmatrix} - \tau = 0.$$

The terms ml^2 , b , and $mg l$ together form the "lumped parameters".

It is worth taking a moment to reflect on this factorization. First of all, it does represent a somewhat special statement about the multibody equations: the nonlinearities enter only in a particular way. For instance, if I had terms in the equations of the form, $\sin(m\theta)$, then I would *not* be able to produce an affine decomposition separating m from θ . Fortunately, that doesn't happen in our mechanical systems [6]. Again, this structure is particular to the *inverse dynamics*, as we have written here. If you were to write the forward dynamics, multiplying by $\mathbf{M}^{-1}$ in order to solve for $\ddot{\mathbf{q}}$, then once again you would destroy this affine structure.

Example 18.4 (Multibody parameters in DRAKE)

Very few robotics simulators have any way for you to access the parameters of the dynamics. In Drake, we explicitly declare all of the parameters of a multibody system in a separate data structure to make them available, and we can leverage Drake's symbolic engine to extract and manipulate the equations with respect to those variables.

As a simple example, I've loaded the cart-pole system model from URDF, created a symbolic version of the `MultibodyPlant`, and populated the `Context` with symbolic variables for the quantities of interest. Then I can evaluate the (inverse) dynamics in order to obtain my equations.

 Launch in Deepnote

The output looks like:

```
Symbolic dynamics:
(0.10000000000000001 * v(0) - u(0) + (pow(v(1), 2) * mp * l * sin(q(1))) + (vd(0) * mc)
(0.10000000000000001 * v(1) - (vd(0) * mp * l * cos(q(1))) + (vd(1) * mp * pow(l, 2)) +
```

Go ahead and compare these with the [cart-pole equations](#) that we derived by hand.

Drake offers a method `DecomposeLumpedParameters` that will take this expression and factor it into the affine expression above. For this cart-pole example, it extracts the lumped parameters $[m_c + m_p, m_p l, m_p l^2]$.

The existence of the lumped-parameter decomposition reveals that the *equation error* for lumped-parameter estimation, with the error taken in the torque coordinates, can again be solved using least squares. Importantly, because we have reduced this to a least-squares problem, we can also understand when it will *not* work. Once again, not all of the lumped-parameters are identifiable, so we will typically need a second step to operate on the identifiable lumped parameters or to add a regularization term.

Example 18.5 (Parameter estimation for the Cart-Pole)

Having extracted the lumped parameters from the URDF file above, we can now take this to fruition. I've kept the example simple: I've simulated the cart-pole for a few seconds running just simple sine wave trajectories at the input, then constructed the data matrix and performed the least squares fit.

The output looks like this:

```
Estimated Lumped Parameters:
(mc + mp). URDF: 11.0, Estimated: 10.905425349337081
(mp * l). URDF: 0.5, Estimated: 0.5945797067753813
(mp * pow(l, 2)). URDF: 0.25, Estimated: 0.302915745122919
```

Note that we could have easily made the fit more accurate with more data (or more carefully selected data).

Should we be happy with only estimating the (identifiable) lumped parameters? Isn't it the true original parameters that we are after? The linear algebra decomposition of the data matrix (assuming we apply it to a sufficiently rich set of data), is actually revealing something fundamental for us about our system dynamics. Rather than feel disappointed that we cannot accurately estimate some of the parameters, we should embrace that *we don't need to estimate those parameters* for any of the dynamic reasoning we will do about our equations (simulation, verification, control design, etc). The identifiable lumped parameters are precisely the subset of the lumped parameters that we need.

18.2.4 Identification using energy instead of inverse dynamics.

In addition to leveraging tools from linear algebra, there are a number of other refinements to the basic recipe that leverage our understanding of mechanics. One important example is the "energy formulations" of parameter estimation [12].

We have already observed that evaluating the equation error in torque space (inverse dynamics) is likely better than evaluating it in state space space (forward dynamics), because we can avoid inverting the mass matrix and stay linear in the lumped parameters. The total energy of the system (kinetic + potential) is also affine in the lumped parameters. We can use the relation

$$\dot{E}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = \dot{\mathbf{q}}^T (\mathbf{B}\mathbf{u} + \text{friction}, \dots).$$

Why might we prefer to work in energy coordinates rather than torque? The differences are apparent in the detailed numerics. In the torque formulation, we find ourselves using $\ddot{\mathbf{q}}$ directly. Conventional wisdom holds that joint sensors can reliably report joint positions and velocities, but that joint accelerations, often obtained by numerically differentiating *twice*, tend to be much more noisy. In some cases, it might be numerically better to apply finite differences to the total energy of the system rather than to the individual joints [†]. [13] provides a study of various filtering formulations which leads to a recommendation in [12] that the energy formulation tends to be numerically superior. This distinction is probably more important in online parameter estimation, where the filters must be causal, as opposed to offline parameter estimation where acausal filters can be used.

[†] Some methods written as numeric integrators power in the different the total but the r should be different

18.2.5 Residual physics models with linear function approximators

The term "residual physics" has become quite popular recently (e.g. [14]) as people are looking for ways to combine the modeling power of deep neural networks with our best tools from mechanics. But actually this idea is quite old, and there is a natural class of residual models that fit very nicely into our least-squares lumped-parameter estimation. Specifically, we can consider models of the form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \alpha_r \Phi(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{B}\mathbf{u} + \text{friction}, \quad \text{etc.}, \quad (4)$$

with α_r the additional parameters of the residual and Φ a set of (fixed) nonlinear basis functions. The hope is that these residual models can capture any "slop terms" in the dynamics that are predictable, but which we did not include in our original parametric model. Nonlinear friction and aerodynamic drag are commonly cited examples.

Common choices for these basis functions, $\Phi(\mathbf{q}, \dot{\mathbf{q}})$, for use with the manipulator equations include radial basis functions[15] or wavelets [16]. Although allowing the basis functions to depend on $\ddot{\mathbf{q}}$ or $\mathbf{u}$ would be fine for the parameter estimation, we tend to restrict them to $\mathbf{q}$ and $\dot{\mathbf{q}}$ to maintain some of the other nice properties of the manipulator equations (e.g. control-affine).

Due to the maturity of least-squares estimation, it is also possible to use least-squares to effectively determine a subset of basis functions that effectively describe the dynamics of your system. For example, in [17], we applied least-squares to a wide range of physically-inspired basis functions in order to make a better ODE approximation of the post-stall aerodynamics during perching, and ultimately discarded all but the small number of basis functions that best described the data. Nowadays, we could apply algorithms like [LASSO](#) for least-squares regression with an ℓ_1 -regularization, or [18] uses an alternative based on sequential thresholded least-squares.

18.2.6 Experiment design as a trajectory optimization

One key assumption for any claims about our parameter estimation algorithms recovering the true identifiable lumped parameters is that the data set was sufficiently rich; that the trajectories were "parametrically exciting". Basically we need to assume that the trajectories produced motion so that the data matrix, $\mathbf{W}$ contains information about all of the identifiable lumped parameters. Thanks to our linear parameterization, we can evaluate this via numerical linear algebra on $\mathbf{W}$.

Moreover, if we have an opportunity to change the trajectories that the robot executes when collecting the data for parameter estimation, then we can design trajectories which try to maximally excite the parameters, and produce a numerically-well conditioned least squares problem. One natural choice is to minimize the [condition number](#) of the data matrix, $\mathbf{W}$. The condition number of a matrix is the ratio of the largest to smallest singular values $\frac{\sigma_{\max}(\mathbf{W})}{\sigma_{\min}(\mathbf{W})}$. The condition number is always greater than one, by definition, and the lower the value the better the condition. The condition number of the data matrix is a nonlinear function of the data taken over the entire trajectory, but it can still be optimized in a nonlinear trajectory optimization ([6], § 12.3.4).

18.2.7 Online estimation and adaptive control

The field of adaptive control is a huge and rich literature; many books have been written on the topic (e.g [19]). Allow me to make a few quick references to that literature here.

I have mostly discussed parameter estimation so far as an offline procedure, but one important approach to adaptive control is to perform online estimation of the parameters as the controller executes. Since our estimation objective can be linear in the (lumped) parameters, this often amounts to the recursive least-squares

estimation. To properly analyze a system like this, we can think of the system as having an augmented state space, $\bar{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ \alpha \end{bmatrix}$, and study the closed-loop dynamics of the state and parameter evolution jointly. Some of the strongest results in adaptive control for robotic manipulators are confined to fully-actuated manipulators, but for instance [20] gives a nice example of analyzing an adaptive controller for underactuated systems using many of the tools that we have been developing in these notes.

As I said, adaptive control is a rich subject. One of the biggest lessons from that field, however, is that one may not need to achieve convergence to the true (lumped) parameters in order to achieve a task. Many of the classic results in adaptive control make guarantees about task execution but explicitly do *not* require/guarantee convergence in the parameters.

18.2.8 Identification with contact

Can we apply these same techniques to e.g. walking robots that are making and breaking contact with the environment?

There is certainly a version of this problem that works immediately: if we know the contact Jacobians and have measurements of the contact forces, then we can add these terms directly into the manipulator equations and continued with the least-squares estimation of the lumped parameters, even including frictional parameters.

One can also study cases where the contact forces are not measured directly. For instance, [21] studies the extreme case of identifiability of the inertial parameters of a passive object with and without explicit contact force measurements.

18.3 IDENTIFYING (TIME-DOMAIN) LINEAR DYNAMICAL SYSTEMS

If multibody parameter estimation forms the first relevant pillar of established work in system identification, then identification of linear systems forms the second. Linear models have been a primary focus of system identification research since the field was established, and have witnessed a resurgence in popularity during just the last few years as new results from machine learning have contributed new bounds and convergence results, especially in the finite-sample regime (e.g. [22, 23, 24, 25]).

A significant portion of the linear system identification literature (e.g. [26]) is focused on identifying linear models in the frequency domain. Indeed, transfer-function realizations provide important insights and avoid some of the foibles that we'll need to address with state-space realizations. However, I will focus my attention in this chapter on time-domain descriptions of linear dynamical systems; some of the lessons here are easier to generalize to nonlinear dynamics (plus we unfortunately haven't built the foundations for the frequency domain techniques yet in these notes).

18.3.1 From state observations

Let's start our treatment with the easy case: fitting a linear model from direct (potentially noisy) measurements of the state. Fitting a discrete-time model, $\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n]$, to sampled data $(\mathbf{u}_n, \mathbf{x}_n = \mathbf{x}[n] + \mathbf{v}[n])$ using the *equation-error* objective is just another *linear* least-squares problem. Typically, we form some data matrices and write our least-squares problem as:

$$\mathbf{X}' \approx [\mathbf{A} \quad \mathbf{B}] \begin{bmatrix} \mathbf{X} \\ \mathbf{U} \end{bmatrix}, \quad \text{where}$$

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_0 & \mathbf{x}_1 & \cdots & \mathbf{x}_{N-2} \\ | & | & & | \end{bmatrix}, \quad \mathbf{X}' = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_{N-1} \\ | & | & & | \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} | & | & & | \\ \mathbf{u}_0 & \mathbf{u}_1 & \cdots & \mathbf{u}_{N-1} \\ | & | & & | \end{bmatrix}$$

By the virtues of linear least squares, this estimator is unbiased with respect to the uncorrelated process and/or measurement noise, $\mathbf{w}[n]$ and $\mathbf{v}[n]$.

Example 18.6 (Cart-pole balancing with an identified model)

I've provide a notebook demonstrating what a practical application of linear identification might look like for the cart-pole system, in a series of steps. First, I've designed an LQR balancing controller, but using the *wrong* parameters for the cart-pole (I changed the masses, etc). This LQR controller is enough to keep the cart-pole from falling down, but it doesn't converge nicely to the upright. I wanted to ask the question, can I use data generated from this experiment to identify a better linear model around the fixed-point, and improve my LQR controller?

Interestingly, the simple answer is "no". If you only collect the input and state data from running this controller, you will see that the least-squares problem that we formulate above is rank-deficient. The estimated $\mathbf{A}$ and $\mathbf{B}$ matrices, denoted $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$ describe the data, but do not reveal the true linearization. And if you design a controller based on these estimates, you will be disappointed!

Fortunately, we could have seen this problem by checking the rank of the least-squares solution. Generating more examples won't fix this problem. Instead, to generate a richer dataset, I've added a small additional signal to the input: $u(t) = \pi_{lqr}(\mathbf{x}(t)) + 0.1 \sin(t)$. That makes all of the difference.

 [Launch in Deepnote](#)

I hope you try the code. The basic algorithm for estimation is disarmingly simple, but there are a lot of details to get right to actually make it work.

Model-based Iterative Learning Control (ILC)

Example 18.7 (The Hydrodynamic Cart-Pole)

One of my favorite examples of model-based ILC was a series of experiments in which we explored the dynamics of a "hydrodynamic cart-pole" system. Think of it as a cross between the classic cart-pole system and a fighter jet (perhaps a little closer to the cartpole)!

Here we've replaced the pole with an airfoil (hydrofoil), turned the entire system on its side, and dropped it into a water tunnel. Rather than swing-up and balance the pole against gravity, the task is to balance the foil in its unstable configuration against the hydrodynamic forces. These forces are the result of unsteady fluid-body interactions; unlike the classic cart-pole system, this time we do not have an tractable parameterized ODE model for the system. It's a perfect problem for system identification and ILC.

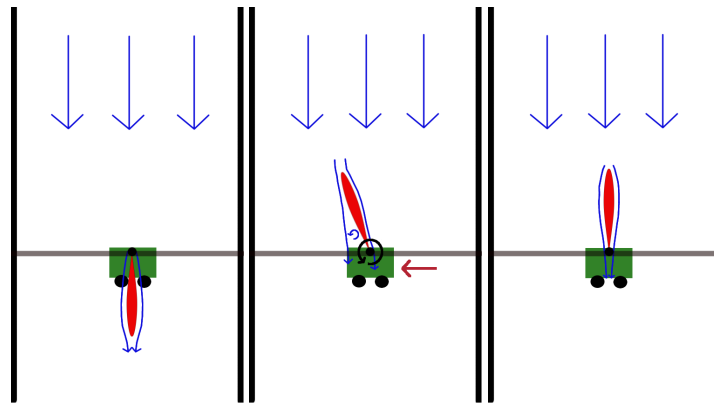


Figure 18.2 - A cartoon of the hydronamic cart-pole system. The cart is actuated horizontally, the foil pivots around a passive joint, and the fluid is flowing in the direction of the arrows. (The entire system is horizontal, so there is no effect from gravity.) The aerodynamic center of the airfoil is somewhere in the middle of the foil; because the pin joint is at the tail, the passive system will "weather vane" to the stable "downward" equilibrium (left). Balancing corresponds to stabilizing the unstable "upward" equilibrium (right). The fluid-body dynamics during the transition (center) are unsteady and very nonlinear.

In a series of experiments, first we attempted to stabilize the system using an LQR controller derived with an approximate model (using [flat-plate theory](#)). This controller didn't perform well, but was just good enough to collect relevant data in the vicinity of the unstable fixed point. Then we fit a linear model, recomputed the LQR controller using the model, and got notably better performance.

To investigate more aggressive maneuvers we considered making a rapid step change in the desired position of the cart (like a fighter jet rapidly changing altitude). Using only the time-invariant LQR balancing controller with a shifted set point, we naturally observed a very slow step-response. Using trajectory optimization on the time-invariant linear model used in balancing, we could do much better. But we achieved considerably better performance by iteratively fitting a time-varying linear model in the neighborhood of this trajectory and performing model-based ILC.

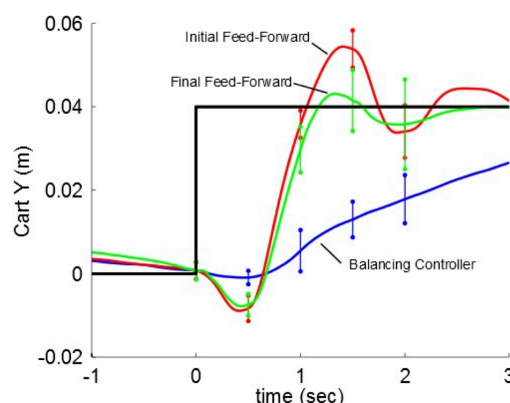


Figure 18.3 - Comparison of the step response using three different controllers: the balancing LQR controller (blue), LQR with a feed-forward term obtained from trajectory optimization with the LTI model (red), and a controller obtained via ILC with a time-varying linear model and iLQR (green).

These experiments were quite beautiful and very visual. They went on from here to consider the effect of stabilizing against incoming vortex disturbances using real-

time perception of the oncoming fluid. If you're at all interested, I would encourage you to check out John Robert's thesis[27] and/or even the [video of his thesis defense](#).

Compression using the dominant eigenmodes

For high-dimensional state or input vectors, one can use singular value decomposition (SVD) to solve this least-squares problem using only the dominant eigenvalues (and corresponding eigenvectors) of $\mathbf{A}$ [28, Section 7.2] using the so-called "Dynamic Mode Decomposition" (DMD). There have been many empirical success stories of using a small number of dominant modes to produce an excellent fit to the data (this is especially relevant in problems like fluid dynamics where the state vector $\mathbf{x}$ might be an entire image corresponding to a fluid flow). In DMD, we would write the linear dynamics in the coordinates of these eigenmodes (which can always be projected back to the full coordinates).

Linear dynamics in a nonlinear basis

A potentially useful generalization that we can consider here is identifying dynamics that evolve linearly in a coordinate system defined by some (potentially high-dimensional) basis vectors $\phi(\mathbf{x})$. In particular, we might consider dynamics of the form $\dot{\phi} = \frac{\partial \phi}{\partial \mathbf{x}} \dot{\mathbf{x}} = \mathbf{A}\phi(x)$. Much ado has been made of this particular form, due to the connection to Koopman operator theory that we will discuss briefly below. For our purposes, we also need a control input, so might consider a form like $\dot{\phi} = \mathbf{A}\phi(x) + \mathbf{B}\mathbf{u}$.

Once again, thanks to the maturity of least-squares, with this approach it is possible to include list many possible basis functions, then use sparse least-squares and/or the modal decomposition to effectively find the important subset.

Note that multibody parameter estimation described above is not this, although it is closely related. The least-squares lumped-parameter estimation for the manipulator equations uncovered dynamics that were still *nonlinear* in the state variables.

18.3.2 From input-output data (the state-realization problem)

In the more general form, we would like to estimate a model of the form

$$\begin{aligned}\mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n] \\ \mathbf{y}[n] &= \mathbf{C}\mathbf{x}[n] + \mathbf{D}\mathbf{u}[n] + \mathbf{v}[n].\end{aligned}$$

Once again, we will apply least-squares estimation, but combine this with the famous "Ho-Kalman" algorithm (also known as the "Eigen System Realization" (ERA) algorithm) [29]. My favorite presentation of this algorithm is [24].

First, observe that

$$\begin{aligned}\mathbf{y}[0] &= \mathbf{C}\mathbf{x}[0] + \mathbf{D}\mathbf{u}[0] + \mathbf{v}[0], \\ \mathbf{y}[1] &= \mathbf{C}(\mathbf{A}\mathbf{x}[0] + \mathbf{B}\mathbf{u}[0] + \mathbf{w}[0]) + \mathbf{D}\mathbf{u}[1] + \mathbf{v}[1], \\ \mathbf{y}[n] &= \mathbf{C}\mathbf{A}^n\mathbf{x}[0] + \mathbf{D}\mathbf{u}[n] + \mathbf{v}[n] + \sum_{k=0}^{n-1} \mathbf{C}\mathbf{A}^{n-k-1}(\mathbf{B}\mathbf{u}[k] + \mathbf{w}[k]).\end{aligned}$$

For the purposes of identification, let's write $\mathbf{y}[n]$ as a function of the most recent $N+1$ inputs (for $k \geq N$):

$$\mathbf{y}[n] = [\mathbf{CA}^{N-1}\mathbf{B} \quad \mathbf{CA}^{N-2}\mathbf{B} \quad \dots \quad \mathbf{CB} \quad \mathbf{D}] \begin{bmatrix} \mathbf{u}[n-N] \\ \mathbf{u}[n-N+1] \\ \vdots \\ \mathbf{u}[n] \end{bmatrix} + \delta[n]$$

$$= \mathbf{G}[n] \bar{\mathbf{u}}[n] + \delta[n]$$

where $\delta[n]$ captures the remaining terms from initial conditions, noise and control inputs before the (sliding) window. $\mathbf{G}[n] = [\mathbf{CA}^{N-1}\mathbf{B} \quad \mathbf{CA}^{N-2}\mathbf{B} \quad \dots \quad \mathbf{CB} \quad \mathbf{D}]$, and $\bar{\mathbf{u}}[n]$ represents the concatenated $\mathbf{u}[n]$'s from time $n - N$ up to n . Importantly we have that $\delta[n]$ is uncorrelated with $\bar{\mathbf{u}}[n]$: $\forall k > n$ we have $E[\mathbf{u}_k \delta_n] = 0$. This is sometimes known as Neyman orthogonality, and it implies that we can estimate $\mathbf{G}$ using simple least-squares $\hat{\mathbf{G}} = \operatorname{argmin}_{\mathbf{G}} \sum_{n \geq N} \|\mathbf{y}_n - \mathbf{G} \bar{\mathbf{u}}_n\|^2$. [24] gives bounds on the norm of the estimation error as a function of the number of samples and the variance of the noise.

How should we pick the window size N ? By Neyman orthogonality, we know that our estimates will be unbiased for any choice of $N \geq 0$. But if we choose N too small, then the term $\delta[n]$ will be large, leading to a potentially large variance in our estimate. For stable systems, the δ terms will get smaller as we increase N . In practice, we choose N based on the characteristic settling time in the data (roughly until the impulse response becomes sufficiently small).

If you've studied linear systems, $\mathbf{G}$ will look familiar; it is precisely this (multi-input, multi-output) matrix impulse response, also known as the "Markov parameters". In fact, estimating $\hat{\mathbf{G}}$ may even be sufficient for control design, as it is closely-related to the parameterization used in disturbance-based feedback for partially-observable systems [30, 31]. But the Ho-Kalman algorithm can be used to extract good estimates $\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{C}}, \hat{\mathbf{D}}$ with state-dimension $\dim(\mathbf{x}) = n_x$ from $\hat{\mathbf{G}}$, assuming that the true system is observable and controllable with order at least n_x and the data matrices we form below are sufficiently rich [24].

It is important to realize that many system matrices, $\mathbf{A}, \mathbf{B}, \mathbf{C}$, can describe the same input-output data. In particular, for any invertible matrix (aka [similarity transform](#)), $\mathbf{T}$, the system matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and $\mathbf{A}', \mathbf{B}', \mathbf{C}'$ with,

$$\mathbf{A}' = \mathbf{T}^{-1}\mathbf{A}\mathbf{T}, \quad \mathbf{B}' = \mathbf{T}^{-1}\mathbf{B}, \quad \mathbf{C}' = \mathbf{C}\mathbf{T},$$

describe the same input-output behavior. The Ho-Kalman algorithm returns a *balanced realization*. A balanced realization is one in which we have effectively applied a similarity transform, $\mathbf{T}$, which makes the controllability and observability Grammians equal and diagonal [28, Ch. 9] and which orders the states in terms of diminishing effect on the input/output behavior. This ordering is relevant for determining the system order and for model reduction.

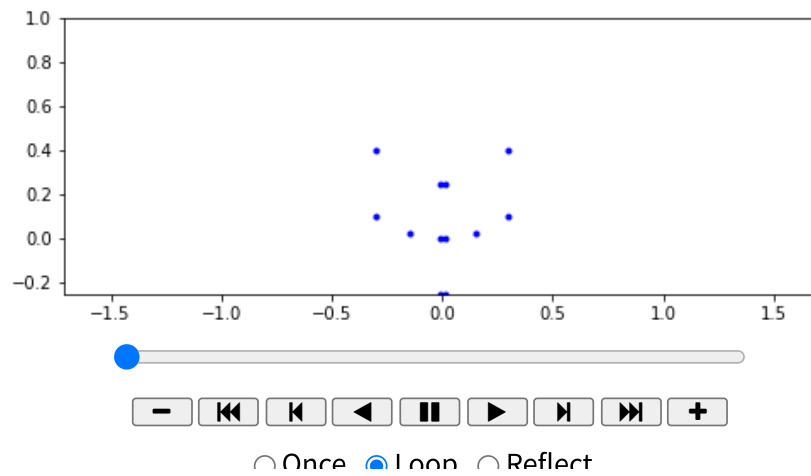
Note that $\hat{\mathbf{D}}$ is the last block in $\hat{\mathbf{G}}$ so is extracted trivially. The Ho-Kalman algorithm tells us how to extract $\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{C}}$, with another application of the SVD on suitably constructed data matrices (see e.g. [24, § 5.1], [32, § 10.5], or [28, § 9.3]).

Example 18.8 (Ho-Kalman identification of the cart-pole from keypoints)

Let's repeat the cart-pole example. But this time, instead of getting observations of the joint position and velocities directly, we will consider the problem of identifying

the dynamics from a camera rendering the scene, but let us proceed thoughtfully.

The output of a standard camera is an RGB image, consisting of 3 x width x height real values. We could certainly columnate these into a vector and use this as the outputs/observations, $y(t)$. But I don't recommend it. This "pixel-space" is not a nice space. For example, you could easily find a pixel that has the color of the cart in one frame, but after an incremental change in the position of the cart, now (discontinuously) takes on the color of the background. Deep learning has given us fantastic new tools for transforming raw RGB images into better "feature spaces", that will be robust enough to deploy on real systems but can make our modeling efforts much more tractable. My group has made heavy use of "keypoint networks" [33] and self-supervised "dense correspondences" [34, 35, 36] to convert RGB outputs into a more consumable form.

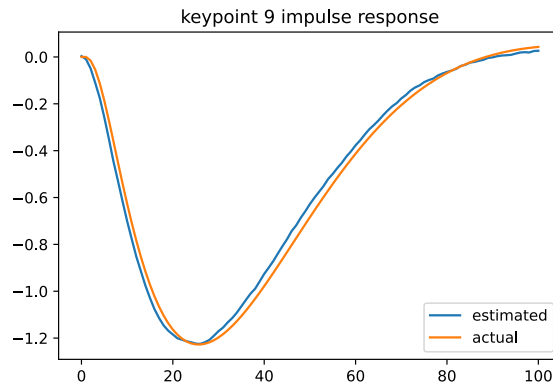


The existence of these tools makes it reasonable to assume that we have observations representing the 2D positions of some number of "key points" that are rigidly fixed to the cart and to the pole. The location of any keypoint K on a pole, for instance, is given by the [cart-pole kinematics](#):

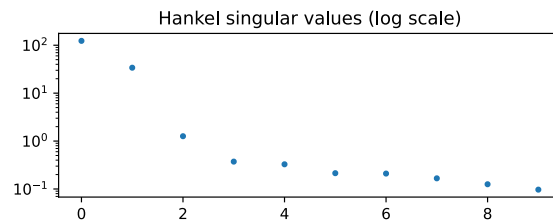
$${}^W\mathbf{p}^K = \begin{bmatrix} x \\ 0 \end{bmatrix} + \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} {}^P\mathbf{p}^K,$$

where x is the position of the cart, and θ is the angle of the pole. I've used [Monogram notation](#) to denote that ${}^P\mathbf{p}^K$ is the Cartesian position of a point A in the pole's frame, P , and ${}^W\mathbf{p}^K$ is that same position in the camera/world frame, W . While it is probably unreasonable to linearize the RGB outputs as a function of the cart-pole positions, x and θ , it *is* reasonable to approximate the keypoint positions by linearizing this equation for small angles.

You'll see in the example that we recover the impulse response quite nicely.



The big question is: does Ho-Kalman discover the 4-dimensional state vector that we know and like for the cart-pole? Does it find something different? We would typically choose the order by looking at the magnitude of the singular values of the Hankel data matrix. In this example, in the case of no noise, you see clearly that 2 states describe most of the behavior, and we have diminishing returns after 4 or 5:



As an exercise, try adding process and/or measurement noise to the system that generated the data, and see how they effect this result.

 Launch in Deeptime

18.3.3 Adding stability constraints

Details coming soon. See, for instance [\[37\]](#).

18.3.4 Autoregressive models

Another important class of linear models predict the output directly from a history of recent inputs and outputs:

$$\mathbf{y}[n+1] = \mathbf{A}_0 \mathbf{y}[n] + \mathbf{A}_1 \mathbf{y}[n-1] + \dots + \mathbf{A}_k \mathbf{y}[n-k] + \mathbf{B}_0 \mathbf{u}[n] + \mathbf{B}_1 \mathbf{u}[n-1] + \dots + \mathbf{B}_k \mathbf{u}[n-k]$$

These are the so-called "AutoRegressive models with eXogenous input (ARX)" models. The coefficients of ARX models can be fit directly from input-output data using linear least squares.

While it is certainly possible to think of these models as state-space models, where we collect the finite history $\mathbf{y}$ and $\mathbf{u}$ (except not $\mathbf{u}[n]$) into our state vector. But this is not necessarily a very efficient representation; the Ho-Kalman algorithm might be able to find a much smaller state representation. This is particularly important for partially-observable systems that might require a very long history, k .

Example 18.9 (An ARX model of cart-pole keypoint dynamics)

18.3.5 Statistical analysis of learning linear models

18.4 IDENTIFICATION OF FINITE (PO)MDPs

There is one more case that we can understand well: when states, actions, and observations (and time) are all discrete. Recall that in the very beginnings of our discussion about optimal control and [dynamic programming](#), we used graph search and discretization of the state space to give a particularly nice version of the value iteration algorithm. In general, we can write the *stochastic* dynamics using conditional probabilities:

$$\Pr(s[n+1]|s[n], a[n]), \quad \Pr(o[n]|s[n], a[n]),$$

where I've (again) used s for discrete states, a for discrete actions, and o for discrete observations. With everything discrete, these conditional probabilities are represented naturally with matrices/tensors. We'll use the tensors $T_{ijk} \equiv \Pr(s[n+1] = s_i | s[n] = s_j, a[n] = a_k)$ to denote the transition matrix, and $O_{ijk} \equiv \Pr(o[n] = o_i | s[n] = s_j, a[n] = a_k)$ for the observation matrix.

18.4.1 From state observations

Following analogously to our discussion on linear dynamical systems, the first case to understand is when we have direct access to state and action trajectories: $s[\cdot], a[\cdot]$. This corresponds to identifying a Markov chain or Markov decision process (MDP). The transition matrices can be extracted directly from the statistics of the transitions:

$$T_{ijk} = \frac{\text{number of transitions to } s_i \text{ from } s_j, a_k}{\text{total number of transitions from } s_j, a_k}.$$

Not surprisingly, for this estimate to converge to the true transition probabilities asymptotically, we need the identification (exploration) dynamics to be [ergodic](#) (for every state/action pair to be visited infinitely often).

For large MDPs, analogous to the modal decomposition that we described for linear dynamical systems, we can also consider factored representations of the transition matrix. [Coming soon...]

18.4.2 Identifying Hidden Markov Models (HMMs)

18.5 NEURAL NETWORK MODELS

If multibody parameter estimation, linear system identification, and finite state systems are the first pillars of system identification, then (at least these days) deep learning is perhaps the last major pillar. It's no coincidence that I've put this section just after the section on linear dynamical systems. Neural networks are powerful tools, but they can be a bit harder to think about carefully. Thankfully, we'll see that many of the basic lessons and insights from linear dynamical systems still apply here. In particular, we can follow the same progression: learning the dynamics function directly from state observations, learning state-space models from input-output data (with recurrent networks), and learning input-output (autoregressive) models with feedforward networks using a history of outputs.

Deep learning tools allow us to quite reliably tune the neural networks to fit our training data. The major challenges are with respect to data efficiency/generalization, and with respect to actually designing planners / controllers based on these models that have such complex descriptions.

18.5.1 Generating training data

One extremely interesting question that arises when fitting rich nonlinear models like neural networks is the question of how to generate the training data. You might have the general sense that we would like data that provides some amount of "coverage" of the state space; this is particularly challenging for underactuated systems since we have dynamic constraints restricting our trajectories in state space.

For multibody parameter estimation [we discussed](#) using the condition number of the data matrix as an explicit objective for experiment design. This was a luxury of using *linear* least-squares optimization for the regression and it takes advantage of the specialized knowledge we have from the manipulator equations about the model structure. Unfortunately, we don't have an analogous concept in the more general case of nonlinear least squares with general function approximators. This approach also works for identifying linear dynamical systems. The challenge is here perhaps considered less severe since for linear models in the noise-free case even data generated from the impulse response is sufficient for identification.

This topic has received considerable attention lately in the context of *model-based reinforcement learning* (e.g. [38]). Broadly speaking, in the ideal case we would like the training data we use for system identification to match the distribution of data that we will encounter when executing the optimal policy. But one cannot know this distribution until we've designed our controller, which (in our current discussion) requires us to have a model. It is a classic "chicken and the egg" problem. In most cases, it speaks to the importance to interleaving system identification and control design instead of the simpler notion of performing identification once and then using the model forevermore.

18.5.2 From state observations

18.5.3 State-space models from input-output data (recurrent networks)

18.5.4 Input-output (autoregressive) models

You might have heard about a neural network model called the Transformer. There are a massive number of online resources where you can [learn about Transformers](#). These started in the natural language processing (NLP) community, where they rapidly replaced recurrent neural network models which were, at the time, struggling to handle very long sentences (trajectories). Transformers get around this problem by switching back to an autoregressive-style model, but they compress the very long history of previous observations into a compact representation using attention mechanisms. This is what now enables large language models, like chat-GPT, to e.g. write an entire essay, "remembering" names/concepts that it used early in the essay correctly very late in the essay.

18.5.5 Particle-based models

18.5.6 Object-centric models

18.5.7 Modeling stochasticity

18.5.8 Control design for neural network models

18.6 ALTERNATIVES FOR NONLINEAR SYSTEM IDENTIFICATION

18.7 IDENTIFICATION OF HYBRID SYSTEMS

18.8 TASK-RELEVANT MODELS

As discussed briefly in the introduction, the standard objective for system identification is to match the input-output behavior of a dynamical system. Certainly if

we can find a model that predicts all outputs perfectly, then this model should be sufficient for making optimal decisions. But when we cannot make perfect predictions, due to limited data and/or limited model representation power, then the relative cost placed on predicting different outputs becomes important. This feels particularly important when the observations come from a camera -- in most cases predicting every pixel in the scene accurately is not the right objective for a robot trying to use the camera to accomplish a particular task.

Fortunately, the problem of learning task-relevant models or "learning state representations" has become a darling of the machine learning community. There are a lot of smart people thinking about the problem and the field has made considerable progress in the last years. In the immense volume of papers, I think there are a few really core ideas.

What makes a good state representation for control? Two defining properties are: 1) it should be observable, and 2) it should be sufficient for making optimal decisions. In other words, if $\mathbf{z}[n]$ is the state of the dynamic controller (which we can think of as an observer of the learned state representation), then we would like that the optimal policy given this state to take the same action as an optimal policy which uses the entire available history:

$$\pi^*(\mathbf{z}[n]) \approx \bar{\pi}^*(\mathbf{y}[n], \mathbf{y}[n-1], \mathbf{y}[n-2], \dots, \mathbf{u}[n-1], \mathbf{u}[n-2], \dots).$$

Certainly the *belief state*, which is by definition a sufficient statistic for predicting future observations, has this property. But much more minimal representations may also be sufficient. So a third, slightly more tenuous property of a good state representation is that we would like it to be tractable for learning, representation, and decision making.

Being sufficient for making optimal control decisions is a difficult objective because it couples the problem of representation learning / system identification with the problem of control design. How can we know what the optimal action is without solving the optimal control problem? This is one reason that imitation learning [35, 39] is a potentially valuable undertaking -- by providing demonstrations of an "optimal" policy, it reduces representation learning back to classical system identification and allows us to focus on the question of state representation without simultaneously solving control design. Other works like [40] attempt use a planner on the learned representation even during learning, and take a loss function requesting that the actions taken from a policy using the current state matches the actions taken from a planner using the representation to optimize multiple steps into the future.

A closely related idea would ask for the representation to be sufficient to predict the optimal value function (rather than the optimal action). Certainly if we have a model and a value function that is consistent with that model, then this would be sufficient for making optimal decisions. But it should be noted that there are cases where predicting the optimal cost-to-go is actually harder than predicting the optimal action. Imagine an agent in a grid world that can traverse one of two corridors and has learned that the left corridor always has more reward than the right. Predicting the precise amount of reward that will be obtained when moving down the left corridor is not needed to choose the optimal action. Predicting optimal values also does not break the "chicken-and-the-egg" problem (how can you know the optimal values until you have an optimal policy?), but may have more subtle merits in terms of representation or learning efficiency.

One proposal which *does* fundamentally break the chicken-and-the-egg problem is given by [41] and called "approximate information states" (AIS). The key idea in this work is that if a state representation, $\mathbf{z}[n]$ is 1) sufficient to predict the one-step reward, and 2) sufficient to predict $\mathbf{z}[n + 1]$, then it is sufficient for making optimal decisions. This motivates their notion of an *approximate* information state, as a representation that approximately accomplishes these two goals. They provide one particular bound for the suboptimality of the resulting policy, based on worst case errors in the one-step reward and next state predictions. In my group, we've been studying this approach, trying to understand the implications for the two cases we can deeply understand: finite (tabular) Markov decision processes [42] and linear-Gaussian systems [43]. In particular, [42] gives an algorithm for solving the AIS conditions to optimality for a class of POMDPs; it provides evidence that representations which are optimal by the AIS metric are not necessarily optimal for reproducing the optimal actions, and that the AIS bounds on performance can be very loose.

Approximate information states are themselves closely related to prior work on "state aggregation" in MDPs via the so-called "bisimulation metrics". The basic idea is that if the future outcomes from two states are indistinguishable, then these states can be aggregated into a single state, thus reducing the state space. See [44] for the original work on exact bisimulation, [45] for approximate bisimulation, [46] for an extension to POMDPs, and [4] for a more modern version using neural networks.

Interestingly, the famous work by DeepMind on MuZero [47] learns a task-relevant state representation using a combination of all of these ideas: the loss function for the state combines a loss for predicting the action, the value function, and the one-step reward. But I have seen a few projects that struggled to reproduce its success in robotics domains.

This area is moving fast; I will try to update this section as results appear, and will give some worked out examples soon.

18.9 EXERCISES

Exercise 18.1 (Linear System Identification with different objective functions)

Consider a discrete-time linear system of the form

$$x[n + 1] = Ax[n] + Bu[n]$$

where $x[n] \in \mathbf{R}^p$ and $u[n] \in \mathbf{R}^q$ are state and control at step n . The system matrix $A \in \mathbf{R}^{p \times p}$ and $B \in \mathbf{R}^{p \times q}$ are unknown parameters of the model, and your task is to identify the parameters given a simulated trajectory. Using the trajectory simulation provided in [this python notebook](#), implement the solution for the following problems.

- Identify the model parameters by solving

$$\min_{A,B} \sum_{n=0}^{N-1} \|x[n + 1] - Ax[n] - Bu[n]\|_2^2.$$

- Identify the model parameters by solving

$$\min_{A,B} \sum_{n=0}^{N-1} \|x[n + 1] - Ax[n] - Bu[n]\|_\infty.$$

c. Identify the model parameters by solving

$$\min_{A,B} \sum_{n=0}^{N-1} \|x[n+1] - Ax[n] - Bu[n]\|_1.$$

Exercise 18.2 (System Identification for the Perching Glider)

In this exercise we will use physically-inspired basis functions to fit the nonlinear dynamics of a perching glider. [In this python notebook](#), you will need to implement least-squares fitting and find the best set of basis functions that describe the dynamics of the glider. Take the time to go through the notebook and understand the code in it, and then answer the following questions. The written question will also be listed in the notebook for your convenience.

- a. Work through the coding sections in the notebook.
- b. All of the basis configurations we tested used at most 3 basis functions to compute a single acceleration. If we increase the number of basis functions used to compute a single acceleration to 4, the least-squares residual goes down. Why would we limit ourselves to 3 basis functions if by using more we can generate a better fit?

REFERENCES

1. Alexander Rakhlin and Karthik Sridharan, "Statistical Learning and Sequential Prediction", Online Draft, September, 2014.
2. Elad Hazan and others, "Introduction to online convex optimization", *Foundations and Trends in Optimization*, vol. 2, no. 3-4, pp. 157--325, 2016.
3. A. Rakhlin, "IDS.160 – Mathematical Statistics: A Non-Asymptotic Approach", Online lecture notes, Spring, 2022.
4. Amy Zhang and Rowan McAllister and Roberto Calandra and Yarin Gal and Sergey Levine, "Learning invariant representations for reinforcement learning without reconstruction", *arXiv preprint arXiv:2006.10742*, 2020.
5. Manuel Watter and Jost Springenberg and Joschka Boedecker and Martin Riedmiller, "Embed to Control: A Locally Linear Latent Dynamics Model for Control from Raw Images", *Advances in Neural Information Processing Systems*, vol. 28, 2015.
6. W Khalil and E Dombre, "Modeling, Identification and Control of Robots", Elsevier, 2004.
7. Christopher G Atkeson and Chae H An and John M Hollerbach, "Estimation of inertial parameters of manipulator loads and links", *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 101--119, 1986.
8. Patrick M Wensing and Sangbae Kim and Jean-Jacques E Slotine, "Linear matrix inequalities for physically consistent inertial parameter identification: A

- statistical perspective on the mass distribution", *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 60--67, 2017.
9. M. Kaess and A. Ranganathan and F. Dellaert, "iSAM: Incremental Smoothing and Mapping", *IEEE Transactions on Robotics*, vol. 24, no. 6, pp. 1365-1378, 2008.
 10. Taeyoon Lee and Patrick M Wensing and Frank C Park, "Geometric robot dynamic identification: A convex programming approach", *IEEE Transactions on Robotics*, vol. 36, no. 2, pp. 348--365, 2019.
 11. Caleb Rucker and Patrick M Wensing, "Smooth parameterization of rigid-body inertia", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2771--2778, 2022.
 12. Maxime Gautier, "Dynamic identification of robots with power model", *Proceedings of the IEEE International Conference on Robotics and Automation*, vol. 3, pp. 1922--1927 vol.3, 1997.
 13. Maxime Gautier, "A comparison of filtered models for dynamic identification of robots", *Proceedings of the 35th IEEE Conference on Decision and Control*, vol. 1, pp. 875--880, 1996.
 14. Andy Zeng and Shuran Song and Johnny Lee and Alberto Rodriguez and Thomas Funkhouser, "Tossingbot: Learning to throw arbitrary objects with residual physics", *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307--1319, 2020.
 15. R. M. Sanner and J. E. Slotine, "Gaussian Networks for Direct Adaptive Control", *1991 American Control Conference*, pp. 2153-2159, 1991.
 16. Robert M Sanner and Jean-Jacques E Slotine, "Structurally dynamic wavelet networks for adaptive control of robotic systems", *International Journal of Control*, vol. 70, no. 3, pp. 405--421, 1998.
 17. Warren Hoburg and Russ Tedrake, "System Identification of Post Stall Aerodynamics for UAV Perching", *Proceedings of the AIAA Infotech@Aerospace Conference*, April, 2009. [[link](#)]
 18. Steven L Brunton and Joshua L Proctor and J Nathan Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems", *Proceedings of the national academy of sciences*, vol. 113, no. 15, pp. 3932--3937, 2016.
 19. Karl J. Åström and Björn Wittenmark, "Adaptive Control: Second Edition", Courier Corporation, apr, 2013.
 20. Joseph Moore and Russ Tedrake, "Adaptive Control Design for Underactuated Systems Using Sums-of-Squares Optimization", *Proceedings of the 2014 American Control Conference (ACC)*, June, 2014. [[link](#)]
 21. Nima Fazeli and Roman Kolbert and Russ Tedrake and Alberto Rodriguez, "Parameter and contact force estimation of planar rigid-bodies undergoing

- frictional contact", *International Journal of Robotics Research*, vol. 36, no. 13-14, pp. 1437-1454, 2017. [[link](#)]
22. Moritz Hardt and Tengyu Ma and Benjamin Recht, "Gradient descent learns linear dynamical systems", *arXiv preprint arXiv:1609.05191*, 2016.
 23. Elad Hazan and Holden Lee and Karan Singh and Cyril Zhang and Yi Zhang, "Spectral filtering for general linear dynamical systems", *arXiv preprint arXiv:1802.03981*, 2018.
 24. Samet Oymak and Necmiye Ozay, "Non-asymptotic identification of lti systems from a single trajectory", *2019 American control conference (ACC)* , pp. 5655-5661, 2019.
 25. Max Simchowitz and Ross Boczar and Benjamin Recht, "Learning linear dynamical systems with semi-parametric least squares", *Conference on Learning Theory* , pp. 2714--2802, 2019.
 26. L. Ljung, "System Identification: Theory for the User", Prentice Hall , 1999.
 27. John W. Roberts, "Control of Underactuated Fluid-Body Systems with Real-Time Particle Image Velocimetry", PhD thesis, Massachusetts Institute of Technology, June, 2012. [[link](#)]
 28. Steven L Brunton and J Nathan Kutz, "Data-driven science and engineering: Machine learning, dynamical systems, and control", Cambridge University Press , 2019.
 29. BL Ho and Rudolf E Kálmán, "Effective construction of linear state-variable models from input/output functions", *at-Automatisierungstechnik*, vol. 14, no. 1-12, pp. 545--548, 1966.
 30. Sadra Sadraddini and Russ Tedrake, "Robust Output Feedback Control with Guaranteed Constraint Satisfaction", *In the Proceedings of 23rd ACM International Conference on Hybrid Systems: Computation and Control* , pp. 12, April, 2020. [[link](#)]
 31. Max Simchowitz and Karan Singh and Elad Hazan, "Improper learning for non-stochastic control", *Conference on Learning Theory* , pp. 3320--3436, 2020.
 32. Jer-Nan Juang and Minh Q. Phan, "Identification and Control of Mechanical Systems", Cambridge University Press , aug, 2001.
 33. Lucas Manuelli* and Wei Gao* and Peter Florence and Russ Tedrake, "kPAM: KeyPoint Affordances for Category-Level Robotic Manipulation", *arXiv e-prints*, pp. arXiv:1903.06684, Mar, 2019. [[link](#)]
 34. Peter R. Florence* and Lucas Manuelli* and Russ Tedrake, "Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation", *Conference on Robot Learning (CoRL)* , October, 2018. [[link](#)]

35. Peter Florence and Lucas Manuelli and Russ Tedrake, "Self-Supervised Correspondence in Visuomotor Policy Learning", *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 492-499, April, 2020. [[link](#)]
36. Lucas Manuelli and Yunzhu Li and Pete Florence and Russ Tedrake, "Keypoints into the Future: Self-Supervised Correspondence in Model-Based Reinforcement Learning", *Conference on Robot Learning (CoRL)*, 2020. [[link](#)]
37. Jack Umenberger and Johan W\aaagberg and Ian R Manchester and Thomas B Schön, "Maximum likelihood identification of stable linear dynamical systems", *Automatica*, vol. 96, pp. 280--292, 2018.
38. Alekh Agarwal and Nan Jiang and Sham M. Kakade and Wen Sun, "Reinforcement Learning: Theory and Algorithms", Online Draft , 2020.
39. Pete Florence and Corey Lynch and Andy Zeng and Oscar A Ramirez and Ayzaan Wahid and Laura Downs and Adrian Wong and Johnny Lee and Igor Mordatch and Jonathan Tompson, "Implicit behavioral cloning", *Conference on Robot Learning*, pp. 158--168, 2022.
40. Aravind Srinivas and Allan Jabri and Pieter Abbeel and Sergey Levine and Chelsea Finn, "Universal planning networks: Learning generalizable representations for visuomotor control", *International Conference on Machine Learning*, pp. 4732--4741, 2018.
41. Jayakumar Subramanian and Aditya Mahajan, "Approximate information state for partially observed systems", *2019 IEEE 58th Conference on Decision and Control (CDC)*, pp. 1629--1636, 2019.
42. Lujie Yang and Kaiqing Zhang and Alexandre Amice and Yunzhu Li and Russ Tedrake, "Discrete Approximate Information States in Partially Observable Environments", *Proceedings of the 2022 American Control Conference (ACC)* , 2022. [[link](#)]
43. Yi Tian and Kaiqing Zhang and Russ Tedrake and Suvrit Sra, "Can Direct Latent Model Learning Solve Linear Quadratic Gaussian Control?", *arXiv preprint arXiv:2212.14511*, 2022.
44. Robert Givan and Thomas Dean and Matthew Greig, "Equivalence notions and model minimization in Markov decision processes", *Artificial Intelligence*, vol. 147, no. 1-2, pp. 163--223, 2003.
45. Norm Ferns and Prakash Panangaden and Doina Precup, "Metrics for Finite Markov Decision Processes.", *UAI*, vol. 4, pp. 162--169, 2004.
46. Pablo Samuel Castro and Prakash Panangaden and Doina Precup, "Equivalence Relations in Fully and Partially Observable Markov Decision Processes.", *IJCAI* , vol. 9, pp. 1653--1658, 2009.
47. Julian Schrittwieser and Ioannis Antonoglou and Thomas Hubert and Karen Simonyan and Laurent Sifre and Simon Schmitt and Arthur Guez and Edward

Lockhart and Demis Hassabis and Thore Graepel and others, "Mastering Atari, Go, chess and shogi by planning with a learned model", *Nature*, vol. 588, no. 7839, pp. 604--609, 2020.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024